When the user submits the form, the raw POST data is sent back to our application. Rails extracts the fields from the form and constructs the `params` hash. Simple values (such as the `id` field, extracted by routing from the form action) are stored directly in the hash. But if a parameter name has brackets in it, Rails assumes that it is part of more structured data and constructs a hash to hold the values. Inside this hash, the string inside the brackets acts as the key. This process can repeat if a parameter name has multiple sets of brackets in it.

Form Parameters	Params
id=123	{ id: "123" }
user[name]=Dave	{ user: { name: "Dave" }}
user[address][city]=Wien	{ user: { address: { city: "Wien" }}}

In the final part of the integrated whole, model objects can accept new attribute values from hashes, which allows us to say this:

```
user.update(user_params)
```

Rails integration goes deeper than this. Looking at the `html.erb` file in the preceding figure, we can see that the template uses a set of helper methods to create the form's HTML; these are methods such as `form_with` and `text_field`.

Before moving on, it's worth noting that `params` may be used for more than text. Entire files can be uploaded. We'll cover that next.